

KernelMind: The Direct Neural Scheduler

Architecting a Deep Reinforcement Learning CPU Dispatcher

Introduction: The Blueprint

In Assignment 3, your Meta-Scheduler learned to orchestrate a fixed menu of traditional heuristics, choosing *when* to call FCFS, SJF, or Round Robin. The agent never touched a process directly; it only picked which hand-written algorithm got to make the decision. That module deliberately deferred the harder question stated in its own introduction: can an agent learn to schedule processes *directly*, with no heuristic standing between its output and the CPU?

This module is that deferred challenge. You will discard the discrete heuristic-selection action space entirely. Your new agent will look directly at the ready queue, at the individual processes, their burst times, and their wait times, and decide, tick by tick, *which specific process* runs next. There is no SJF or RR hiding inside the action space anymore. If the agent wants SJF-like behavior, it has to discover it from scratch, balanced against a fairness signal you design into the reward. This is a substantially harder learning problem than Assignment 3's tabular Q-table over a handful of discretized states, and it will require a function approximator, a neural network, in place of a Q-table, since the state space (the raw contents of the queue) is no longer something you can bucket into a handful of integers without throwing away the information the agent actually needs.

1 Part 1: Architecture & Direct State Representation

Your Assignment 3 process representation, workload generator, and baseline simulators (FCFS, SJF, Round Robin, Random) are almost entirely reusable here. The heuristics themselves do not change. What changes is what the *agent* is allowed to see and do.

Why a Q-Table Stops Working

In Assignment 3, you discretized the queue into a handful of integer buckets (e.g. queue length 0–3, 4–7, 8+) precisely so a tabular Q-table could index into it. That worked because the agent only needed a coarse summary of the queue to decide which *heuristic* to invoke; the heuristic itself handled the fine-grained process-by-process decision.

Now the agent *is* the heuristic. It needs to compare individual processes against each other, this one's burst time against that one's wait time against a third one's priority, to decide which single process to run. Bucketing that into a table would require a table with one row per possible *combination* of process attributes across every queue slot, which explodes combinatorially. You need a function approximator that can generalize across queue contents it has never seen in exactly that configuration before. That is what the neural network buys you.

Your Task:

1. **Process Representation:** Extend your Assignment 3 `Process` class with a `virtual_runtime` field, the cumulative ticks this specific process has actually been executed for. You will need this for the fairness mechanism in Part 3; tracking accumulated service time, not just wait time, is

what lets your reward function distinguish “has been waiting a while” from “has actually been starved of CPU time.”

2. **Workload Generation:** Reuse your Assignment 3 generator. No changes are required for the core assignment.
3. **State Tensor:** Write a function that converts the top N processes in the ready queue (e.g. $N = 10$) into a fixed-size tensor of per-process features, at minimum normalized remaining burst time, normalized wait time, and normalized priority. Pad unused slots with zeros.
4. **Action Masking:** Because the ready queue will frequently contain fewer than N processes, you must also produce a boolean validity mask alongside the state tensor, marking which slots are real processes and which are padding. Your network’s output for padded slots must be forced to $-\infty$ before any `argmax`, or your agent will eventually try to schedule a process that does not exist.

Design Question 1: The Cost of Direct Control

Assignment 3’s agent picked from k heuristics, where k was perhaps 4 or 5. Your new agent effectively picks from up to N raw processes every tick, where N is your queue window size. Why does this larger, per-tick action space make the credit-assignment problem harder for the learning algorithm? Specifically: if the agent makes one bad process-selection decision early in a 60-tick episode, how many subsequent reward signals have to correctly propagate that blame backward before the network’s weights reflect it, and why is that propagation chain longer here than it was when the agent only made one heuristic-selection decision per several ticks of RR execution?

2 Part 2: The Evaluation Metrics

Your Assignment 3 metrics suite, Mean Wait Time, P90 Wait Time, and Jain’s Fairness Index, carries over unchanged. The formulas do not care whether the schedule was produced by a heuristic or a neural network.

Your Task: Confirm your existing `calculate_metrics` utility operates correctly on the output of a direct-control episode (a list of completed processes with populated `arrival_time`, `finish_time`, and `burst_time`). No new metrics are required for the core assignment, but you are encouraged to log P90 wait time throughout training, not only at final evaluation; it will surface failure modes that Mean Wait Time alone can hide.

3 Part 3: Formulating the Direct-Control Environment

This is the heart of the assignment. You are no longer building an MDP whose actions are heuristics; you are building one whose actions are processes.

The Discounted Completion Spike, and Why It Can Backfire

A naive first attempt at a reward function looks tempting: give the agent a large bonus the instant a process completes, and a small constant penalty every other tick to discourage dawdling. On paper this seems to reward urgency. In practice, every tick that *is not* a completion can end up producing nearly the same penalty regardless of which process the agent picked, leaving little gradient to distinguish a good choice from a bad one until a job finally finishes, many ticks later and heavily discounted. If your completion bonus is also an order of magnitude larger than your per-tick shaping terms, the network's value estimates can end up dominated almost entirely by "ticks until the next completion," and whatever fairness signal you intended gets drowned out by the dominant gradient. If your agent mysteriously converges to something that looks exactly like FCFS no matter how you change your reward weights, check whether your state features are actually populated (a zeroed-out feature carries no information regardless of its intended purpose) and whether your reward scale lets every term you wrote meaningfully compete for gradient influence, not just the largest one.

Your Task:

1. **State Space:** Implement `reset()` and `step(action)` on a `DirectSchedulerEnvironment` class. `step` should: execute the selected process for one tick, advance the clock, admit newly-arrived processes into the ready queue, and return the next state tensor, next mask, reward, and a `done` flag.
2. **Action Space:** The action is simply an integer index into the visible ready queue. Validate at the top of `step` that the chosen index is actually marked valid in the current mask; an agent that ever selects a padded slot has a bug in its masking, not a bug worth silently tolerating.
3. **Reward Function, Fairness via Aging:** Implement a CFS-style fairness mechanism that mathematically echoes the Aging concept from Assignment 3: track each process's `virtual_runtime`, and penalize the agent each tick for the *variance* of virtual runtime across the current ready queue. This rewards keeping service spread evenly, independent of how long any individual process has been waiting.
4. **Reward Function, Starvation Cap:** Consider layering a second, separate penalty on top of the variance term: once any single process's wait time exceeds a fixed threshold, apply a penalty that escalates with how far past the threshold it is. Whatever escalation curve you choose, treat it as a hypothesis to test, not a guaranteed fix; see Part 4's note on diagnosing before tuning.
5. **Bound Every Unbounded Term:** Whatever penalty terms you design, make sure none of them can grow without limit. A reward term that is unbounded will occasionally spike to a magnitude many times larger than your completion bonus under a random exploring policy, and a term that can dominate the scale of everything else on a sizeable fraction of steps will destabilize training regardless of how carefully you tuned the weight in front of it.

Design Question 2: Reward Shaping, Revisited

Assignment 3 asked you to explain how your reward function prevented an agent from justifying the indefinite starvation of one massive job to save aggregate wait time elsewhere. Answer it again for your direct-control reward function, however you ended up designing it.

Then consider the following scenario, whether or not it matches what you actually observed: a fairness term computed from raw *wait time* (“penalize the system for how long the longest-waiting process has been waiting”) can create a different incentive than a fairness term computed from *virtual runtime* (“penalize the system for uneven accumulated service”), because rescuing a starved long job under a wait-time penalty means inheriting many additional ticks of “this job is still in the queue” penalty while it executes, since everyone else’s wait keeps climbing while it runs. If you used a wait-time-based penalty and did not observe this effect, explain why your specific design avoided it. If you used a virtual-runtime-based penalty from the start, explain why you expect it to be less susceptible to this trap in the first place. Either answer is acceptable; what matters is that your reasoning is grounded in your own reward function, not a result you are assuming you must have hit.

4 Part 4: The Direct Scheduler Network

Build a neural network to replace your Assignment 3 Q-table.

Approximating Attention Over the Ready Queue

A simple feedforward network that flattens the entire state tensor into one long vector will technically work, but it forces the network to learn the *position* of useful information rather than its *content*: it has no inherent way to know that “process in slot 3” and “process in slot 7” should be compared the same way regardless of which slot either one happens to occupy. A self-attention mechanism, by contrast, lets the network compare every visible process against every other visible process directly, independent of queue position, which is closer to what a real scheduling decision requires: ranking candidates against each other, not against a fixed slot index.

Your Task:

1. **Network Architecture:** Implement a `DirectSchedulerNet` that projects each process’s feature vector into an embedding dimension, adds a positional encoding, passes the result through a multi-head self-attention layer, and produces a single Q-value per visible process from a small feedforward head.
2. **Masking Inside the Network:** Use your validity mask as the `key_padding_mask` for the attention layer, and separately force the *output* Q-values of padded slots to $-\infty$ before they are ever passed to an `argmax`. These are two different masking operations serving two different purposes; confirm in your report why both are necessary and what would go wrong if you implemented only one of them.
3. **Bellman Update with Double DQN:** Implement the standard DQN training loop, experience replay, a target network, an ϵ -greedy policy with decay, but use *Double DQN* for the target computation: select the best next action using your policy network, then evaluate that action’s value using the target network. Using the same network for both selection and evaluation is a

well-documented source of value overestimation, since any noise that makes one action's value spike gets picked by `argmax`, and the same network is then asked to confirm its own spike.

4. **Diagnose Before You Tune:** Before touching any reward weight, log the mean magnitude of your Q-values across training, in chunks (e.g. every 50 episodes). A healthy, converging Q-function should plateau at a magnitude roughly consistent with your actual reward scale and discount factor. If you instead see a Q-function that keeps climbing for the entire training run, especially while the agent's actual episode reward stays flat, that is a sign of divergence, and no amount of reward-weight tuning will fix a divergence problem, because the issue lives in the Bellman update, not the reward formula. You may or may not encounter this; either way, log it so you know which case you are in.

Design Question 3: Diagnosing Instability

This question is conditional. If your Q-value logging from Part 4 and your seed-to-seed results from Part 5 both look stable and well-behaved, briefly describe what you checked and why you are confident the agent is not just getting lucky on a narrow set of seeds, and you are done with this question.

If instead you find that identical hyperparameters produce noticeably different final fairness scores across different random seeds, for example sometimes clearly beating SJF and sometimes landing no better than it, do not stop at "DQN is just noisy, nothing to be done." Identify at least two specific, checkable quantities you logged or could log (candidates include: the raw magnitude of any single reward term under a random policy, how frequently your target network refreshes relative to gradient steps taken, or the signed, not absolute, mean TD-error over training) and explain what result from each would tell you "this is a fixable bug" versus "this is inherent noise I have to live with."

5 Part 5: Mission Control & Comparative Benchmarking

Your Task:

1. Re-run your Assignment 3 baseline evaluations (FCFS, SJF, RR, Random) on a fixed universal test set of workloads, recording Mean Wait Time and Jain's Fairness Index for each.
2. Train your direct-control agent for several hundred episodes. Track Mean Wait Time per episode for your convergence plot, exactly as in Assignment 3.
3. **Convergence Plot:** Plot a moving average of the agent's Mean Wait Time across training episodes, overlaid with flat horizontal reference lines for each baseline, identical in spirit to your Assignment 3 plot.
4. **Final Comparison Table:** Evaluate your fully-trained agent (zero exploration) against the same universal test set used for the baselines, and produce a table reporting Mean Wait Time and Jain's Fairness Index for all five policies side by side.
5. **Tradeoff Visualization:** Produce a second plot, a scatter of Mean Wait Time (x-axis) against Jain's Fairness Index (y-axis), with one point per policy. This should make visually clear whether your agent's point sits strictly inside the Pareto frontier defined by SJF and FCFS (a real, justified tradeoff) or whether it is dominated by one of the baselines (worse on both axes than something that took one line of code to write).

Design Question 4: Output Analysis

Analyze your final comparative table and both plots. Does your agent's point on the tradeoff scatter plot represent an improvement over SJF's fairness for a bounded, reasonable wait-time cost, or is it dominated by SJF, FCFS, or RR on one or both axes, or does it land somewhere you would describe differently from either of those two framings? All three outcomes are possible depending on your specific reward design and training run, and none of them is automatically a failure to report honestly.

If your result is dominated, or only ties an existing one-line heuristic, say so plainly, and describe what you would try next to address it, whether that is a reward-weight adjustment, a training-stability fix, or something else. A result that merely matches a baseline is not, by itself, evidence your agent learned anything useful beyond what the baseline already provides; justify your conclusion with the actual numbers in your table, not just the shape of your convergence plot.

Deliverables

Submit your codebase archive containing the following components (or encapsulated in a single notebook):

1. The extended process representation and (reused) workload generator.
2. The Direct-Control MDP Environment, including your fairness reward logic.
3. The `DirectSchedulerNet` architecture and the Double DQN training loop.
4. The evaluation and metrics suite (reused from Assignment 3).
5. The training execution script demonstrating your training loop.
6. A generated convergence plot and a generated tradeoff scatter plot.
7. A comprehensive `report.pdf` explaining your code implementation, your answers to the 4 Design Questions, and an honest breakdown of how your agent's final performance compares to the baselines, including any instability you encountered during training and how, or whether, you resolved it.

Optional Objectives

Symmetric Multiprocessing (Multi-Core Scheduling)

Your environment so far simulates a single CPU core. Real systems have not shipped with one core in decades.

The Challenge: Expand your environment to feature 4 or 8 independent CPU cores. Rather than selecting one process per tick, your agent must now effectively fill up to C cores simultaneously. A genuine joint action space over which-process-goes-on-which-core is combinatorial and intractable for a single scalar-per-action DQN output; a more tractable approach is to have your network score every visible process exactly once, then greedily assign the top- C ranked, distinct processes to the C cores. Track which core each process last executed on, and add a small reward bonus for keeping a process on the same core it last ran on, and a matching penalty for migrating it, so your agent can naturally discover a preference for *CPU affinity* without you ever hand-coding the rule. Re-run your full baseline-vs-agent comparison with multi-core-aware versions of FCFS, SJF, and RR (all routed through the same core-assignment logic as your agent, so the comparison stays fair), and report a migrations-per-completed-job metric alongside your usual Mean Wait Time and fairness figures.

Memory/Resource Constraints

Right now, every process that arrives is immediately schedulable. A real system has finite RAM, and a process that does not fit cannot run no matter how short its burst time is.

The Challenge: Assign a `memory_required` attribute to each process and give your environment a hard total-memory cap. Processes that arrive but do not currently fit should sit in a secondary *swap queue* rather than entering the ready queue. Admission from swap into the ready queue should be a deterministic first-fit policy that runs automatically every tick, *not* an additional decision your agent has to make, which keeps your action space unchanged while still making memory pressure a real, consequential part of the simulation. Track average ticks-spent-in-swap as a new metric. Does your agent, without ever being told memory exists as a concept it should optimize for, end up with a lower average swap time than the baselines simply because it clears memory-resident jobs more efficiently? Report whether this holds, and explain in your report whether you think that result is a coincidence of your reward function or a genuine emergent consequence of it.

The Workload Stress Test

Currently your workload generator likely distributes burst times and arrivals somewhat uniformly.

The Challenge: Create a secondary workload mode called “I/O Storm,” generating 8 incredibly short jobs (bursts of 1–2 ticks) arriving simultaneously alongside 2 massive CPU-heavy jobs (bursts of 50+ ticks). Evaluate your trained network, trained only on uniform workloads, against this skewed distribution without retraining. Does its fairness index collapse, or does the learned policy generalize? If it collapses, is the failure visible in the state features your network was given, or is it a genuine out-of-distribution generalization gap?

Seed Stability as a First-Class Metric

Design Question 3 asked you to diagnose training instability if you encountered it. This objective asks you to quantify it properly, whether or not you found a problem to fix.

The Challenge: Train your agent from at least 5 independent random seeds with identical hyperparameters. Report not just the mean Mean-Wait-Time and mean fairness across seeds, but their *standard deviation*, and the fraction of seeds whose final result is strictly dominated by plain SJF (worse on both Mean Wait Time and fairness simultaneously). A reward function and training setup that only “works” on a lucky seed is not a solved problem; it is an unsolved one wearing a single good result as a disguise. If your spread turns out to be wide, identify and fix the actual source of the instability rather than training many seeds and quietly keeping only the best one.